

Predicting User Sentiment in Software Issue Reports

ST2MLE — Machine Learning for IT Engineers

A pipeline to classify issue-report sentiment: negative · neutral · positive

Comparing vectorisation strategies and classifiers, with full tuning, imbalance handling, dimensionality reduction, pruning and early stopping.

Problem & objectives

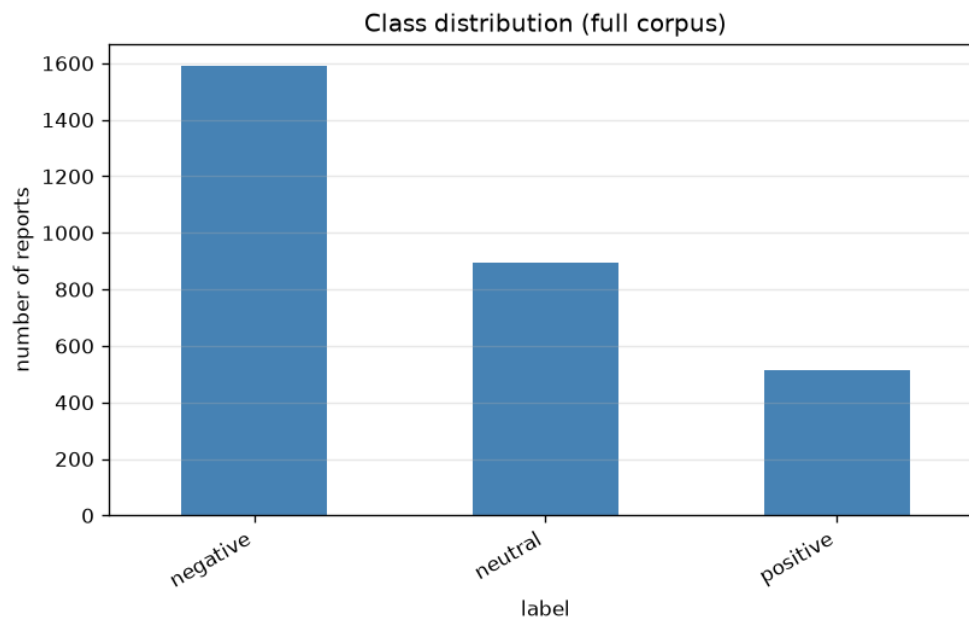
- **Goal:** automatically classify the sentiment of user-reported software issues (bug reports, feature requests, reviews) into **negative / neutral / positive**.
- **Why it matters:** triage angry reports faster, spot satisfaction trends, prioritise fixes.
- **What we compare:**
 - 4 vectorisers — Bag-of-Words, TF-IDF, Word2Vec, pre-trained GloVe (+ BERT code)
 - 5 classifiers — Naive Bayes, Decision Tree, Random Forest, AdaBoost, Logistic Regression
- **And we study:** class imbalance (SMOTE), PCA, tree pruning, early stopping.

Dataset

- **3 000 synthetic issue reports** built to mirror real trackers (reproducible, seed = 42).
- **Deliberately imbalanced** — complaints dominate:

negative	neutral	positive
53 %	30 %	17 %

- **Made non-trivial on purpose:** shared software vocabulary across classes, 22 % tone-neutral/ambiguous reports, 15 % mixed-signal openers/tails, 8 % label noise → realistic ceiling ≈ 0.88 macro-F1.



Pipeline overview

raw text → preprocess → vectorise → [balance] → [reduce] → classify → evaluate

- **Preprocess:** lower-case, strip punctuation/URLs, tokenise, drop stop-words (**keep negations**), WordNet lemmatise.
- **Vectorise:** BoW · TF-IDF · Word2Vec · pre-trained GloVe.
- **Balance (train only):** SMOTE / under-sampling via `imblearn`.
- **Reduce:** TruncatedSVD (PCA for sparse text).
- **Classify & tune:** 5 models, 5-fold CV, `GridSearchCV`.
- **Evaluate:** accuracy, macro P/R/F1, confusion matrix.

Engineering: preprocess once + cache vectorisers → whole study in ~4-5 min.

Preprocessing

- **Clean:** lower-case → remove URLs/HTML/punctuation/digits.
- **Tokenise** then **remove stop-words** — but **keep negations** (not , no , never ...) because they flip sentiment.
- **Lemmatise** (WordNet, verb-then-noun): crashing → crash, issues → issue.
- Implemented as a stateless scikit-learn transformer → no train/test leakage.

Example "Critical issue -- The app keeps crashing!!!" → critical issue app keep crash

Vectorisation — 4 strategies

Strategy	Idea	Output
Bag-of-Words	n-gram counts	sparse, ≥ 0
TF-IDF	counts $\times$ inverse document frequency	sparse, ≥ 0
Word2Vec	mean of embeddings trained on our corpus	dense
GloVe (pre-trained)	mean of Wikipedia/Gigaword 100-d vectors	dense

- Sparse + non-negative $\rightarrow$ feeds **MultinomialNB**; dense $\rightarrow$ **GaussianNB**.
- **BERT** vectoriser is implemented & documented; needs the model hub (unavailable here) so it is skipped automatically.

Models & tuning

- **Naive Bayes** — `MultinomialNB` (sparse) / `GaussianNB` (dense), auto-selected.
- **Decision Tree** — pre-pruning (`max_depth`, `min_samples_leaf`) **and** post-pruning (`ccp_alpha`).
- **Random Forest** — bagged trees.
- **AdaBoost** — boosted stumps.
- **Logistic Regression** — demonstrates **L2 regularisation** (tuned `C`).

All tuned with `GridSearchCV`, **5-fold cross-validation**, scoring **macro-F1**.

Experimental setup

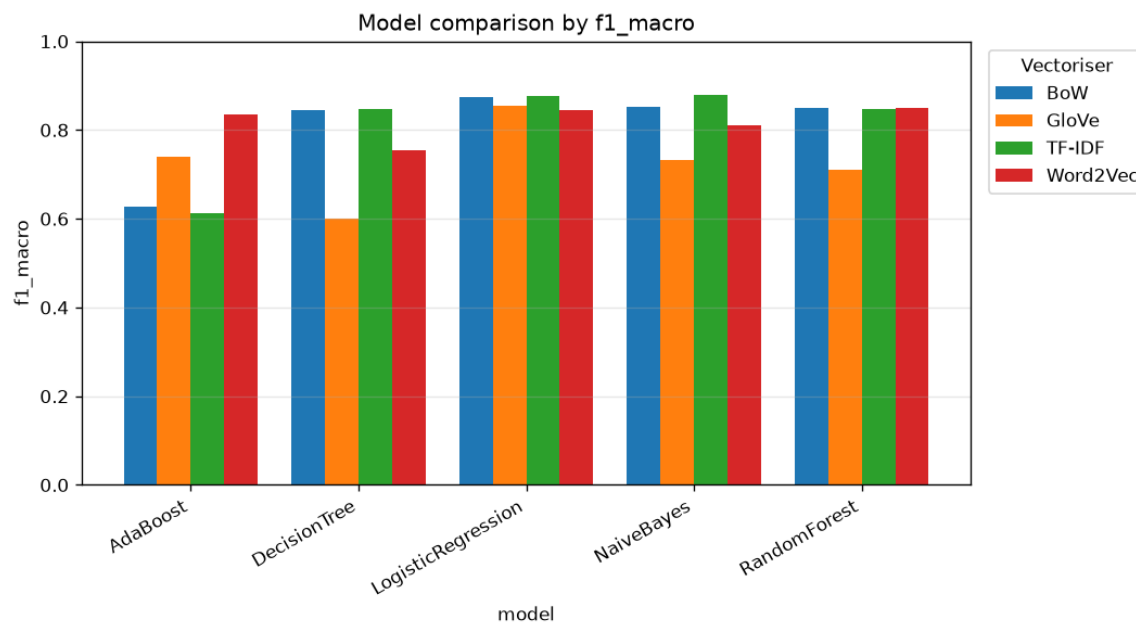
- **Split:** stratified 80 % train / 20 % test.
- **CV:** 5-fold `StratifiedKFold` on the training set.
- **Tuning target:** macro-F1 (weights every class equally on imbalanced data).
- **Reproducibility:** single seed (42); deterministic — a clean re-run reproduces every number byte-for-byte.
- **Compute:** 4 cores; full study $\approx$ 4-5 min thanks to cached vectorisers.

Results — model comparison

Best classifier per vectoriser (test macro-F1):

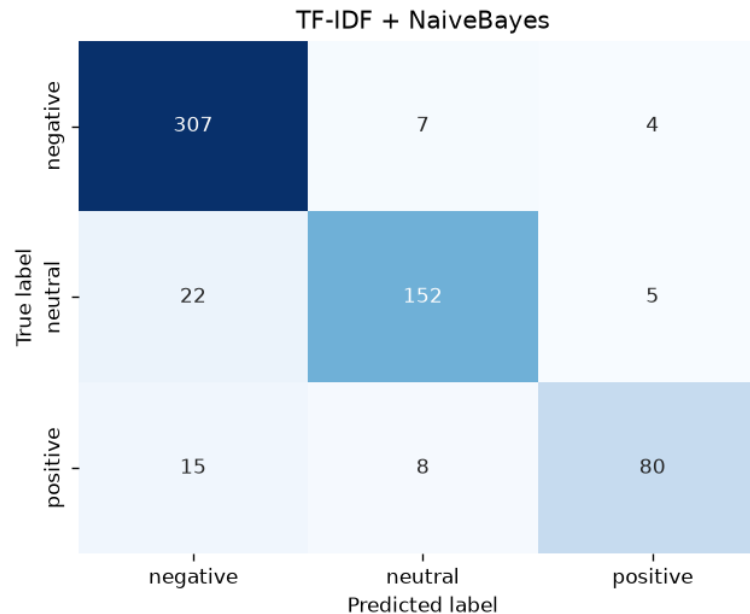
Vectoriser	Best classifier	Macro-F1
TF-IDF	Naive Bayes	0.880
BoW	Logistic Regression	0.874
GloVe (pre-trained)	Logistic Regression	0.856
Word2Vec	Random Forest	0.849

AdaBoost on sparse features is worst (0.61). Full 20-row table in the report.



Results — best model

TF-IDF + Naive Bayes — macro-F1 0.880, accuracy 0.898 — confusion matrix on the held-out test set:



- Errors concentrate on the **ambiguous / noise** reports — the intended ceiling.
- Minority **positive** class is hardest (fewest examples, most overlap).

Class-imbalance handling

TF-IDF + Logistic Regression; only the resampling strategy changes (train folds only).

Strategy	Macro-F1	Recall neg	Recall neu	Recall pos
none	0.877	0.969	0.838	0.767
SMOTE	0.863	0.931	0.860	0.786
under-sampling	0.834	0.887	0.855	0.786

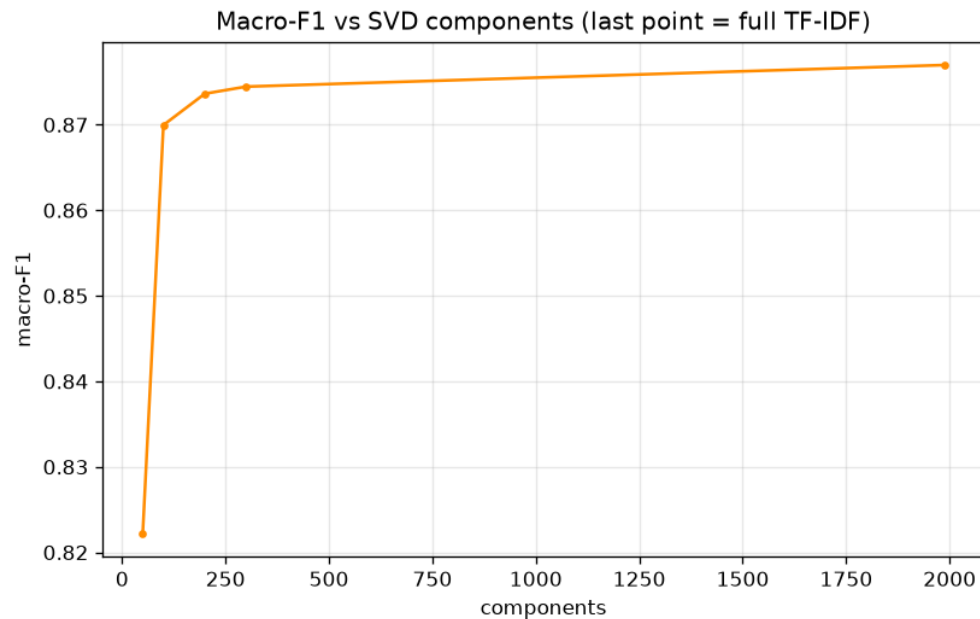
Takeaway: SMOTE **raises minority recall** (positive 0.767 → 0.786) but lowers majority recall → resampling redistributes errors rather than adding accuracy.

Dimensionality reduction (PCA / TruncatedSVD)

TF-IDF (1 987 features) → TruncatedSVD; downstream macro-F1 (Logistic Regression):

Components	Variance	Macro-F1
100	63 %	0.870
200	72 %	0.874
1 987 (full)	100 %	0.877

Takeaway: 200 components (~10 % of features) recover ~99.6 % of full performance — big speed/memory win, essential before dense models.

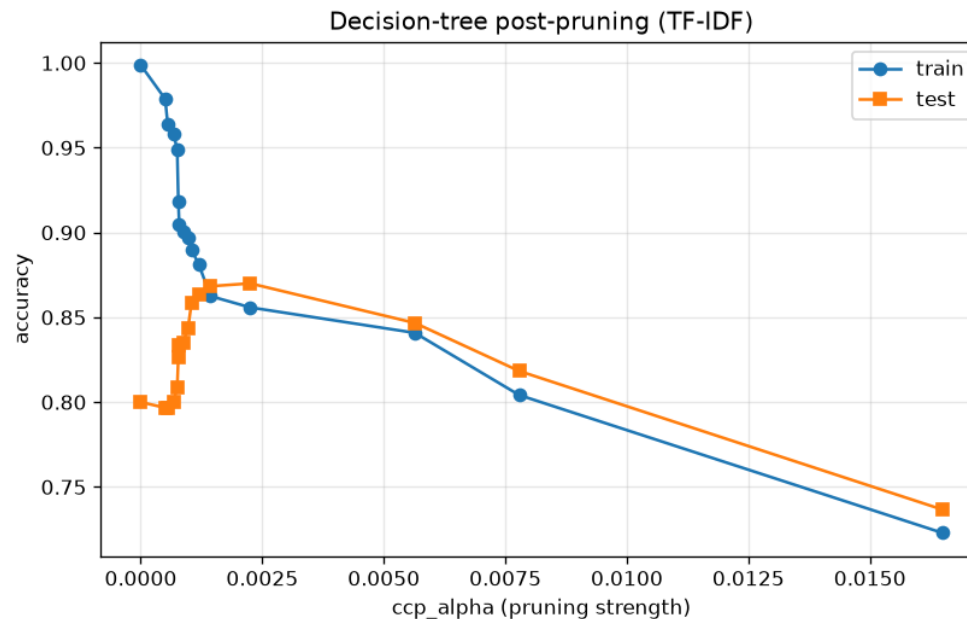


Decision-tree pruning

Cost-complexity (`ccp_alpha`) post-pruning on TF-IDF:

Tree	# nodes	Train acc	Test acc
unpruned	707	0.999	0.800
best pruned	65	0.856	0.870
over-pruned	19	0.723	0.737

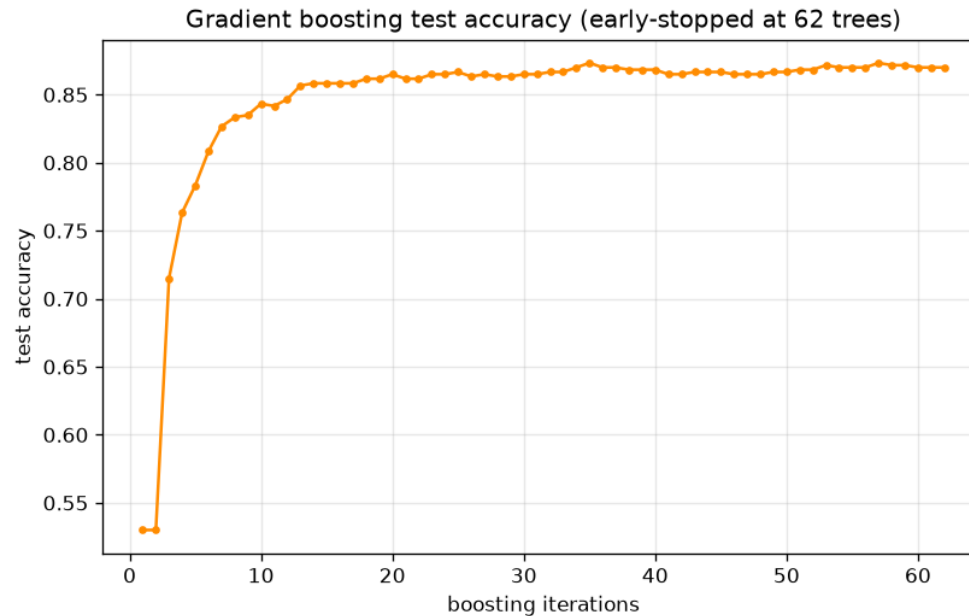
Takeaway: pruning cuts 707 → 65 nodes, **raises test accuracy 0.80 → 0.87**, and closes the over-fitting gap — textbook bias-variance.



Early stopping (gradient boosting)

TF-IDF → SVD(100) → Gradient Boosting, ceiling 500 trees, `n_iter_no_change=10`.

- **Stopped after 62 of 500 trees** — $\sim 8\times$ less training, no loss of test quality.
- Validation monitoring halts once added trees stop helping → guards over-fitting.



Discussion — trade-offs

- **TF-IDF + simple models win** on short, keyword-driven issue text.
- **AdaBoost** collapses on sparse high-dim features (stumps see 1 word of ~5 000) but **recovers on dense embeddings** — match the model to the feature geometry.
- **Pre-trained GloVe** trails TF-IDF here: general-domain vectors + mean-pooling blur the precise lexical sentiment cues of this domain.
- **Resampling** trades majority for minority recall; **PCA** buys efficiency; **pruning & early stopping** both curb over-fitting.

Challenges & future work

Challenges - No downloadable labelled corpus → built a calibrated reproducible generator. - MultinomialNB rejects negatives → auto-switch to GaussianNB for embeddings. - Tuning runtime → preprocess-once + cached vectorisers.

Future work - Run on a **real** GitHub/JIRA corpus (pipeline already accepts any text,label CSV). - Enable **BERT** / fine-tuned transformers. - **Class-weighting** and cost-sensitive thresholds as leakage-free alternatives.

Conclusion

- Built a **complete, reproducible** sentiment pipeline for software issues.
- Compared **4 vectorisers × 5 classifiers**, fully tuned with 5-fold CV.
- **Best: TF-IDF + Naive Bayes (macro-F1 0.880)**, with an honest account of trade-offs.
- Demonstrated imbalance handling, PCA, pruning and early stopping end to end.

Thank you — questions?